Localizing INT8 Quantization Collapse in EfficientNet-B0 for Edge Poultry Disease Detection

Fatih Jawwad Al Mumtaz
Department of Informatics
University of Darussalam Gontor
Email: fatihalmumtaz76@student.cs.unida.gontor.ac.id

Amirul Al Aziz
Department of Agricultural Industrial Technology
University of Darussalam Gontor
Email: amirulalaziz57@student.tip.unida.gontor.ac.id

Muhammad Iqbal Hidayatullah
Department of Informatics
University of Darussalam Gontor
Email: iqbalhidayatullah4563@gmail.com

***Abstract*—Poultry fecal disease detection with deep learning achieves high accuracy, but prior work largely targets cloud inference with heavy models that are impractical for edge use. This paper presents a systematic characterization of three lightweight convolutional neural network architectures—MobileNetV2, ShuffleNetV2, and EfficientNet-B0—for classifying poultry fecal images into four disease categories: Coccidiosis, Salmonellosis, Newcastle Disease, and Healthy. We use the Machuve *et al.* dataset via HuggingFace `Dianyo/poultry-fecal-fl`. We audit the publisher claim of 8,770 deduplicated leak-free images and find 617 near-duplicate pairs (393 exact), including 316 groups spanning train/test and 95 test images byte-identical to train images; all numbers in this paper are therefore reported on a deduplicated sterile split (8,153 images: 5,088/1,337/1,728 train/val/test, zero cross-split survivors). Each model is evaluated under INT8 post-training quantization (ONNX Runtime) on WSL x86 reference and a Raspberry Pi 5 (Cortex-A76, ONNX Runtime 1.29.0, 12 thread/batch configs). All three models exceed 97% FP32 test accuracy on the sterile test set (EfficientNet-B0 98.55% [97.97, 99.07]; MobileNetV2 97.92% [97.22, 98.55]; ShuffleNetV2 97.69% [96.93, 98.38]; 95% bootstrap CI, B=10,000). Our thesis is scheme $\times$ runtime $\times$ architecture with diagnosis: dynamic INT8 preserves MobileNetV2 (97.11%) and ShuffleNetV2 (97.45%) but collapses EfficientNet-B0 to 10.42% (mode collapse to Newcastle Disease); static QDQ per-tensor reproduces the collapse (9.32%) while per-channel recovers to 94.56% (McNemar with continuity correction vs dynamic: b=14, c=1468, p underflows in double precision ($p < 10^{-15}$)), and selective per-family ablation confirms the cause: per-channel scaling on depthwise layers alone recovers 93.46%, while per-channel scaling on every family except depthwise collapses to 8.33%. On RPi5, *dynamic* INT8 is 1.9–2.8$\times$ slower than FP32 at 4 threads and scales by only 1.10–1.19$\times$ (1$\rightarrow$4 threads) versus 2.20–2.33$\times$ for FP32, but *static* QDQ reverses this: per-channel static INT8 reaches 94.56% accuracy at 13.50 ms, 2.34$\times$ faster than FP32 (31.53 ms) on the same board and runtime. Granularity, not bit width, is the decision variable. Interpretability was inspected qualitatively and is not reported as a finding. Together, the results guide hardware-aware model and quantization-scheme choice for edge CPU deployment.**

***Index Terms*—deep learning, poultry disease detection, lightweight CNN, quantization, edge computing, post-training quantization**

I. Introduction

Poultry production is a major agricultural sector in the United States (combined broiler, egg, turkey, and chicken value of $67.1 billion in 2023 [10]). Coccidiosis, Salmonellosis, and Newcastle Disease (NCD) cause major losses [2]. Fecal image analysis enables noninvasive early detection, but laboratory diagnosis remains inaccessible to many small farms [3].

Recent deep learning approaches have achieved high accuracy for poultry fecal disease classification. Dhungana *et al.* reported 99.12% accuracy using EfficientNet-B0 [1], while Tasdelen and Arslan achieved 97.1% with MobileNetV2 [3]. However, these solutions primarily target cloud-based deployment, leaving a critical gap: *how can these models be efficiently deployed on resource-constrained edge devices at farm level?*

Lightweight CNNs—MobileNetV2 [5], ShuffleNetV2 [6], and EfficientNet [7]—are candidates for edge deployment because they need fewer operations. INT8 post-training quantization (PTQ) reduces size by using 8-bit weights [8]. Even on CPUs that expose INT8 acceleration, PTQ can increase latency rather than reduce it when the runtime kernel path adds quantize/dequantize work around every convolution; our x86 reference (12th Gen i5-12400F, AVX-VNNI present per `/proc/cpuinfo`) still shows a 3.0–16.8$\times$ slowdown, so the slowdown is consistent with the ORT CPU kernel path adding quantize/dequantize work, not raw ISA absence; kernel-level attribution is discussed cautiously in Sec. IV-C. See also the quantization white paper for why PTQ without calibration degrades on depthwise layers [14].

This paper addresses two research questions: (1) What is the quantization sensitivity of each lightweight architecture under INT8 PTQ on CPU, and where exactly does EfficientNet-B0 fail? (2) What is the accuracy-latency tradeoff for these models under INT8 PTQ on a Raspberry Pi 5? We additionally audit the dataset integrity claim underlying prior work on this benchmark.

The main contributions of this paper are summarized as follows. First, we audit the FecalFed leak-free claim [9] and show 617 near-duplicate pairs (393 byte-exact) including 95 test images byte-identical to train images; all numbers are reported on a deduplicated sterile split (8,153 images: 5,088/1,337/1,728, zero cross-split survivors). Second,

we localize the EfficientNet-B0 INT8 collapse to per-tensor depthwise quantization, causally rather than by correlation: dynamic INT8 (10.42%) and static per-tensor QDQ (9.32%) fail identically while per-channel QDQ recovers to 94.56%; a selective per-family ablation shows depthwise layers are both necessary (per-channel everywhere except depthwise collapses to 8.33%) and sufficient (per-channel on depthwise alone recovers 93.46%), and layerwise weight SQNR identifies why (27.97 dB per-tensor, +16.46 dB recoverable) (McNemar with continuity correction vs dynamic: b=14, c=1468, p underflows in double precision ($p < 10^{-15}$); vs FP32 $p = 1.37 \times 10^{-13}$). Third, we characterize the sterile models on Raspberry Pi 5 (12 configurations) with node-expansion and kernel-level profiling, showing *dynamic* INT8 1.93–2.80× slower at 4 threads with 1.10–1.19× scaling versus 2.20–2.33× for FP32; a controlled static-QDQ comparison on the same board then shows the opposite direction (2.34× faster than FP32 at 94.56% accuracy), isolating quantization granularity rather than bit width as the operative variable.

II. Related Work

A. Poultry Disease Detection

Machuve *et al.* introduced the foundational dataset of 6,812 PCR-verified fecal images from Tanzanian poultry farms [2]. Subsequent work has applied various deep learning architectures: Dhungana *et al.* combined YOLOv11 detection with EfficientNet-B0 classification, achieving 99.12% accuracy and 25.8 ms inference [1]. Tasdelen and Arslan benchmarked six architectures, finding MobileNetV2 optimal for both accuracy (97.1%) and training speed [3]. Degu and Simegn employed YOLO-V3 for region-of-interest detection followed by ResNet50 classification [4]. Chi introduced FecalFed, a federated-learning benchmark on this dataset family claiming 8,770 rigorously deduplicated unique images [9]; our audit in Section III-A tests that claim directly. Prior work on this dataset has not compared lightweight models under quantization with accuracy-latency tradeoff analysis.

B. Lightweight CNN Architectures

MobileNetV2 [5] utilizes inverted residuals with linear bottlenecks and depthwise separable convolutions for efficient feature extraction. ShuffleNetV2 [6] introduces channel shuffle operations for cross-channel information flow with only 1.26M parameters. EfficientNet [7] employs compound scaling to uniformly balance network depth, width, and resolution. Hoang *et al.* recently compared seven lightweight architectures for plant disease detection, identifying hybrid designs as optimal for edge deployment [11].

C. Model Quantization

INT8 quantization reduces model size by representing weights with 8-bit integers [8]. While theoretically achieving 4× compression, ONNX Runtime documentation [12] explicitly notes that INT8 without matching kernel support may degrade performance due to quantize/dequantize overhead. The white paper on quantization surveys why PTQ without calibration degrades on depthwise and SE layers [14].

III. Methodology

A. Dataset

We use the Machuve *et al.* dataset [13] with PCR-verified fecal images from Tanzanian farms, as distributed via HuggingFace `Dianyo/poultry-fecal-fl`. Chi [9] describes this benchmark as 8,770 rigorously deduplicated leak-free images with train/test splits of 7,016/1,754. We audit this claim with pHash (Hamming ≤ 5) over all 8,770 files and find 617 near-duplicate pairs (393 byte-exact), including 316 groups spanning train/test splits: 95 test images byte-identical to train images and 30 to validation images.

All numbers in this paper are therefore reported on a deduplicated sterile split: we drop one copy per duplicate pair (keeping test $>$ val $>$ train), yielding 8,153 unique images with 5,088/1,337/1,728 train/val/test and zero cross-split survivors under this audit (audit artifact `results/phash_dedup.json`, split manifest `results/sterile_split_manifest.json`).
Concretely: the audit found 617 near-duplicate pairs (all size 2, class-consistent; 393 byte-exact), of which 316 groups span splits; we drop exactly one copy per pair keeping test $>$ val $>$ train (within a split, lexicographically-first kept), so counts refer to removed copies rather than connected groups. The validation split was carved as a stratified 80/20 split of the publisher train only (seed 42); publisher test was never re-split. We use the shorthand *sterile split* throughout for this pHash-deduplicated partition. The term denotes only the absence of cross-split near-duplicate pairs detected by the audit described above; it is not a claim of independence between images, since residual dependence may remain within a split. We note as a limitation that this removes leakage but does not rebuild a fresh split from the Zenodo originals (6,812 images); the sterile test set remains 98% inherited from the publisher test split. Classes are Coccidiosis, Healthy, Salmonella, and Newcastle Disease; NCD is underrepresented (417/8,153 train, 141/1,728 test), so we report per-class INT8 accuracy and bootstrap confidence intervals. Images are resized to 224×224 and normalized with ImageNet mean/std.

B. Model Architectures

We evaluate three ImageNet [16]-pretrained lightweight architectures with the classification head replaced for four classes (3.4M→2.2M for MobileNetV2 after the head swap; parameter counts below are for our 4-class models). MobileNetV2 [5] (2.2M parameters) uses inverted residuals with depthwise separable convolutions for efficient local feature extraction. ShuffleNetV2 [6] (1.3M parameters) relies on channel shuffle for cross-channel mixing with few parameters. EfficientNet-B0 [7] (4.0M parameters) applies compound scaling with squeeze-and-excitation blocks to capture multi-scale features.

C. Training Protocol

All models are ImageNet-pretrained (torchvision V1 weights) with only the classification head replaced (4 classes); all layers are trainable (no freezing). Each model is trained

once (seed 42) for 50 epochs using Adam (learning rate 10^{-4}, weight decay 10^{-4}) with cosine annealing; the checkpoint with the best validation accuracy is kept. Batch size is 32, input resolution 224×224. Data augmentation (train only): random horizontal flip, rotation ($\pm 15°$), color jitter (brightness/contrast/saturation 0.2); eval preprocessing is resize→tensor→ImageNet normalize. Training is performed on an NVIDIA RTX 4060 GPU with 8 GB VRAM using PyTorch 2.5.1 with CUDA 12.1.

D. Quantization and Benchmarking (x86)

Models are exported to ONNX (opset 13, `torch.onnx.export`) and quantized with ONNX Runtime dynamic per-tensor INT8 (`quantize_dynamic`, `weight_type=QInt8`, default MatMul-only op types; CPUExecutionProvider, no calibration dataset; weights INT8, activations dynamically quantized at runtime). No graph fusion settings are changed. For EfficientNet-B0 we additionally run static QDQ quantization (`quant_pre_process` + `quantize_static`, `QuantFormat.QDQ`, MinMax calibration on the first 100 sterile-train images sequentially, batch 8, eval preprocessing; `activation_type=QInt8`, `weight_type=QInt8`) in per-tensor and per-channel variants to diagnose the collapse. The x86 reference latency is measured on a 12th Gen Intel i5-12400F CPU via WSL (CPUExecutionProvider, default intra-op threads; ONNX Runtime 1.29.0, ONNX 1.22.0) with 200 runs and 10 warmup iterations per model, batch 1, model-only timing (eval preprocessing and data transfer excluded); we report mean, P50, P95, P99, throughput (FPS), and model size. These x86 numbers are a reference only, not deployment claims. Power is not measured on x86 or RPi5 (board current telemetry unavailable), so energy is a stated limitation.

E. Edge Deployment on Raspberry Pi 5

We characterize the same ONNX FP32 and INT8 models on a Raspberry Pi 5 (Broadcom BCM2712, 4× Cortex-A76 at up to 2.4 GHz, 8 GB LPDDR4X, Debian 13 trixie, kernel 6.18.34+rpt-rpi-2712, aarch64, `ondemand` governor, throttled=0x0). ONNX Runtime 1.29.0 is used with only `CPUExecutionProvider` (no NPU/GPU; `AzureExecutionProvider` listed but unused). The sterile-model sweep (`scripts/benchmark_pi_gate0.py`) covers 12 configurations: 3 models × 2 quantizations (FP32/INT8 dynamic) × 2 thread counts (1 and 4 via `intra_op_num_threads`) × batch 1, with input 3×224×224. Node counts are measured directly on the sterile ONNX graphs; a Level-2 ORT kernel-timing artifact (10 runs, batch=1, 1 thread) is available at `results/profiling/level2_profiling_artifact.json` (supplementary). Each configuration runs 200 iterations after 10 warmup iterations (single session per config), reporting mean/std/P50/P95/P99 latency, throughput, RSS memory (mean/peak), CPU%, and die temperature before/after. Variability reported is within-session; variability across independent benchmark sessions was not measured. All timings are model-only (eval preprocessing and data transfer excluded), not end-to-end application throughput. Initial temperature was 44.6°C; peak during the sweep was 55.6°C. Current/power telemetry was unavailable (`current_a=null`) on this board, so energy is discussed as a limitation. All 12/12 runs completed without thermal throttling.

IV. Results and Discussion

A. Classification Performance

Table I summarizes the classification performance on the sterile test set (1,728 images). All three architectures achieve over 97% accuracy, demonstrating the effectiveness of ImageNet transfer learning for the poultry fecal domain. We report 95% bootstrap confidence intervals (B=10,000, seed 42). FP32 vs INT8 intervals overlap for MobileNetV2 and ShuffleNetV2 (drops 0.81pp and 0.24pp); overlapping marginal intervals alone do not establish paired significance or equivalence, so we make no significance claim for these two drops without a paired test. The EfficientNet-B0 collapse (88.13pp) is unambiguous.

TABLE I
TEST ACCURACY ON STERILE SPLIT (1,728 IMAGES) WITH 95% BOOTSTRAP CI (B=10,000). DYNAMIC INT8 PTQ (ORT). MACRO COLUMNS ARE MACRO-AVERAGED PRECISION/RECALL/F1 AND BALANCED ACCURACY.

Model	FP32 acc (%)	INT8 acc (%)	Drop (pp)	FP32 macro-F1 (bal. acc)	INT8 macro-F1 (bal. acc)
MobileNetV2	97.92 [97.22, 98.55]	97.11 [96.30, 97.86]	0.81	97.57 (97.26)	96.45 (96.09)
ShuffleNetV2	97.69 [96.93, 98.38]	97.45 [96.64, 98.15]	0.24	97.10 (97.34)	96.82 (96.76)
EfficientNet-B0	98.55 [97.97, 99.07]	10.42 [8.97, 11.92]	88.13	98.22 (98.03)	7.49 (26.89)

EfficientNet-B0 achieves the highest FP32 accuracy (98.55%), but its INT8 accuracy collapses to 10.42%: the confusion matrix shows 1,676/1,728 predictions falling in the Newcastle Disease column (per-class: cocci 2.27%, healthy 4.76%, salmo 0.54%, ncd 100%), a mode collapse rather than gradual degradation. We diagnose this with static QDQ quantization (Table II): per-tensor MinMax reproduces the collapse (9.32%), while per-channel MinMax recovers to 94.56% (McNemar with continuity correction vs dynamic: b=14, c=1468, p underflows in double precision ($p < 10^{-15}$); vs FP32 $p = 1.37 \times 10^{-13}$), consistent with per-tensor depthwise sensitivity rather than squeeze-and-excitation alone. Why granularity, not depthwise structure alone, governs the outcome: per-tensor quantization assigns one shared scale/zero-point to every weight in a tensor, so a depthwise kernel—whose weights span all C output channels with typically C≤32 and no spatial reduction to average over—has far less opportunity for outlier cancellation than a dense pointwise kernel spanning C×K×K weights. EfficientNet-B0 compounds this with squeeze-and-excitation, whose two 1D convolutions and sigmoid/hard-swish gates sit in the dynamic range of a handful of channels, so a per-tensor range estimated from all channels is dominated by whichever channel is largest. MobileNetV2

and ShuffleNetV2 contain the same depthwise-separable primitives but pair them with wide pointwise stages whose weight distributions concentrate more tightly, which is consistent with their surviving per-tensor quantization. Table III isolates this causally rather than by correlation. Quantizing depthwise layers per-channel while leaving every other family per-tensor recovers accuracy to 93.46%, whereas quantizing every family per-channel *except* depthwise collapses to 8.33%. These two configurations are exact complements, so depthwise layers are both necessary and sufficient for the collapse: they are the only family whose per-tensor scaling is individually fatal, and granting them per-channel scales rescues the network while granting per-channel scales to everything else does not. Leaving depthwise entirely in FP32 recovers to 97.16%, while leaving the squeeze-and-excitation convolutions in FP32 still collapses (12.44%), ruling out SE as the primary cause. The mechanism is visible in the weights: depthwise tensors have the worst per-tensor SQNR of any family (27.97 dB mean over 182k parameters) and the largest gain from per-channel scaling (+16.46 dB), because a depthwise kernel holds one filter per output channel and offers no averaging to absorb an outlier, whereas the 3.1M-parameter pointwise layers average over $C \times K \times K$ weights. "Per-class accuracy" in this paper means recall (true-positive rate per true class). Macro-averaged precision/recall/F1 and balanced accuracy for all three architectures under both quantizations are reported in Table I. Under INT8, MobileNetV2 and ShuffleNetV2 retain macro-F1 within 1.1 and 0.3 points of FP32 respectively, while EfficientNet-B0 drops to macro-F1 7.49; NCD, the minority class (141/1,728 test), degrades most under INT8 for all three architectures.

TABLE II
EFFICIENTNET-B0 STATIC QDQ INT8 DIAGNOSIS (MINMAX, CALIBRATION: FIRST 100 STERILE-TRAIN IMAGES, SEQUENTIAL). PER-TENSOR REPRODUCES THE COLLAPSE; PER-CHANNEL RECOVERS. MCNEMAR WITH CONTINUITY CORRECTION.

Config	Test acc. (%)	McNemar vs FP32 p (discord. b/c)
Dynamic INT8	10.42	—
Static QDQ per-tensor	9.32	$< 10^{-15}$ (underflow; b=36, c=17 vs dynamic)
Static QDQ per-channel	94.56	1.37×10^{-13} (b=75, c=7 vs FP32)

TABLE III
SELECTIVE PER-FAMILY STATIC QDQ ABLATION, EFFICIENTNET-B0 (MINMAX, FIRST 100 STERILE-TRAIN IMAGES, QDQ, BATCH 1). GROUPING IS STRUCTURE-DERIVED FROM THE ONNX GRAPH; SE EXPORTS AS BARE `FC1`/`FC2` CONVOLUTIONS WITH NO "SE" SUBSTRING. FAMILIES: 16 DEPTHWISE, 32 POINTWISE, 16 SE-REDUCE, 16 SE-EXPAND, 1 STEM. FP32 REFERENCE IS 98.50%.

	Configuration	Size (MB)	Acc. (%)	Collapse
A	All per-tensor	4.29	9.32	yes
B	All per-channel	4.67	94.56	no
C	**Depthwise** per-channel, rest per-tensor	4.33	**93.46**	no
D	All per-channel *except* depthwise	4.51	8.33	yes
E	Depthwise left in FP32	4.69	97.16	no
F	Squeeze-and-excitation left in FP32	5.91	12.44	yes

TABLE IV
LAYERWISE WEIGHT SQNR UNDER SIMULATED INT8, BY FAMILY (DB, MEAN OVER FAMILY). "GAIN" IS THE IMPROVEMENT PER-CHANNEL SCALING WOULD DELIVER OVER PER-TENSOR. DEPTHWISE IS SIMULTANEOUSLY THE WORST-AFFECTED FAMILY UNDER PER-TENSOR AND THE MOST-RECOVERABLE UNDER PER-CHANNEL, WHICH IS WHY ITS GRANULARITY ALONE DECIDES THE OUTCOME. WEIGHT SQNR IS COMPUTED OFFLINE FROM THE EXPORTED FP32 WEIGHTS; IT DOES NOT DEPEND ON THE CALIBRATION SET.

Family	Tensors	Params	Per-tensor	Per-channel (gain)
Depthwise	16	182k	**27.97**	44.42 (+16.46)
SE reduce	16	314k	32.68	37.14 (+4.46)
SE expand	16	314k	34.00	45.26 (+11.26)
Pointwise	32	3.15M	33.67	42.30 (+8.63)
Stem	1	864	35.19	45.28 (+10.09)

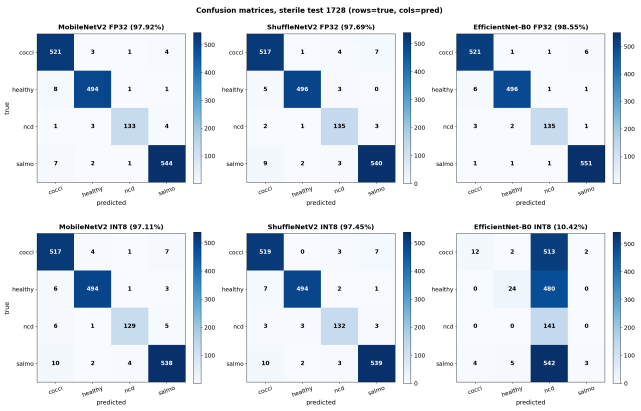


Fig. 1. Confusion matrices on the sterile test set (1,728 images). Top: FP32; bottom: INT8 dynamic. EfficientNet-B0 INT8 (bottom right) collapses to the Newcastle Disease column.

B. Quantization Impact

Table V presents the quantization analysis on the sterile split. All architectures achieve 3.3–3.7× model size reduction through INT8 quantization.

TABLE V
INT8 QUANTIZATION ANALYSIS (WSL X86 REFERENCE, BATCH 1, 200 RUNS + 10 WARMUP, MODEL-ONLY TIMING, DEFAULT THREADS). STERILE-SPLIT MODELS.

Model	Size (MB)		Latency (ms)		Slowdown
	FP32	INT8	FP32	INT8	(INT8/FP32)
MobileNetV2	8.48	2.30	2.40	40.25	16.8×
ShuffleNetV2	4.89	**1.47**	4.15	12.62	3.0×
EfficientNet-B0	15.3	4.16	4.35	57.18	13.1×

INT8 quantization increases latency by 3.0–16.8× versus FP32 on the x86 reference. Our reference CPU exposes AVX-VNNI yet still slows down, suggesting the ORT CPU kernel path (quantize/dequantize ops around the matmul) dominates; graph operators alone do not prove which kernels execute, so Sec. IV-C attributes this cautiously using executed-kernel profiling on RPi5.

ShuffleNetV2 degrades least (3.0× versus 16.8× for MobileNetV2). Channel-shuffle ops are non-parametric and less

sensitive to precision, while MobileNetV2 depthwise separable convolutions are more sensitive.

C. Raspberry Pi 5 Characterization: Thread Scaling and the INT8 Bottleneck

Table VII summarizes sterile-model RPi5 latency at batch=1. FP32 scales with threads: MobileNetV2 drops from 33.16 ms (1 thread, 30.2 FPS) to 14.23 ms (4 threads, 70.3 FPS, 2.33× speedup); ShuffleNetV2 from 14.19 ms to 6.19 ms (2.29×, 161.5 FPS); EfficientNet-B0 from 69.45 ms to 31.56 ms (2.20×). The sweep covers threads 1 and 4; intermediate thread counts are not measured for the sterile models.

Critically, the INT8 slowdown reported in Table VII and Fig. 4 is a property of *dynamic* quantization on this runtime, not of INT8 in general. Table VI repeats the EfficientNet-B0 measurement on the same board, same ORT build, and same session with the static QDQ graphs whose accuracy we characterized in Table II. Static QDQ is 2.31–2.34× *faster* than FP32 at 4 threads (13.63 and 13.50 ms versus 31.53 ms), reversing the direction of the dynamic result (63.99 ms, 0.49× of FP32). The practical consequence is that a deployment choosing per-channel static QDQ obtains both the recovered accuracy (94.56%, Table II) and a genuine latency win, whereas the dynamic graphs trade a 2× latency loss for an accuracy collapse that no runtime setting can repair. Granularity, not bit width, is therefore the decision variable that matters for this model.

TABLE VI
EFFICIENTNET-B0 ON RASPBERRY PI 5: DYNAMIC VS STATIC QDQ INT8, BATCH=1, MEAN OF 200 RUNS AFTER 10 WARMUP, SINGLE SESSION PER CONFIGURATION, CPUEXECUTIONPROVIDER ONLY, ORT 1.29.0, PEAK 49.4°C, THROTTLED=0X0. ACCURACY IN THE LAST COLUMN IS MEASURED ON THE EXPORTED ONNX GRAPHS RATHER THAN THE PYTORCH CHECKPOINTS REPORTED IN TABLE I, WHICH IS WHY THE FP32 REFERENCE READS 98.50% HERE VERSUS 98.55% THERE; THE 0.05-POINT GAP IS ONE IMAGE AND REFLECTS FLOAT32 EXPORT. PER-CHANNEL STATIC QDQ IS BOTH THE ACCURATE AND THE FAST CONFIGURATION.

Quantization	Granularity	Size (MB)	t=1 (ms)	t=4 (ms)	Rel. FP32 t=4	Acc. (%)
FP32	—	15.30	69.00 (68.97)	31.53 (30.89)	1.00×	98.50
INT8 dynamic	per-tensor	4.16	76.38 (76.35)	63.99 (63.82)	0.49×	10.42
INT8 static	per-tensor	4.29	24.51 (24.50)	13.63 (13.37)	2.31×	9.32
INT8 static	**per-channel**	4.67	24.66 (24.64)	**13.50** (13.49)	2.34×	94.56

TABLE VII
RASPBERRY PI 5 LATENCY, BATCH=1 (MEAN OF 200 RUNS, P50 IN PARENTHESES). STERILE-SPLIT MODELS, ORT 1.29.0, 44–56°C, THROTTLED=0X0.

Model	Quant	t=1 (ms)	t=4 (ms)	Scaling 1→4	FP32→INT8 t=4
MobileNetV2	FP32	33.16 (33.00)	**14.23** (14.03)	2.33×	—
	INT8	43.88 (43.86)	39.83 (39.73)	1.10×	2.80× slower
ShuffleNetV2	FP32	14.19 (14.19)	**6.19** (6.18)	2.29×	—
	INT8	13.65 (13.64)	11.95 (11.76)	1.14×	1.93× slower
EfficientNet-B0	FP32	69.45 (69.42)	**31.56** (31.18)	2.20×	—
	INT8	76.68 (76.67)	64.39 (64.19)	1.19×	2.04× slower

INT8 on RPi5 is slower than FP32 at 4 threads (1.93–2.80×), and its thread scaling stalls: from 1→4 threads MobileNetV2 INT8 improves only 1.10× (43.88→39.83 ms), ShuffleNetV2 1.14×, EfficientNet-B0 1.19×, versus 2.20–2.33× for FP32. This indicates limited parallelization of INT8 kernels in ONNX Runtime 1.29.0 on Cortex-A76. One exception: ShuffleNetV2 INT8 at 1 thread (13.65 ms) is slightly faster than its FP32 counterpart (14.19 ms), the only configuration where INT8 wins—consistent with its homogeneous channel-shuffle design and lowest node expansion (1.37×).

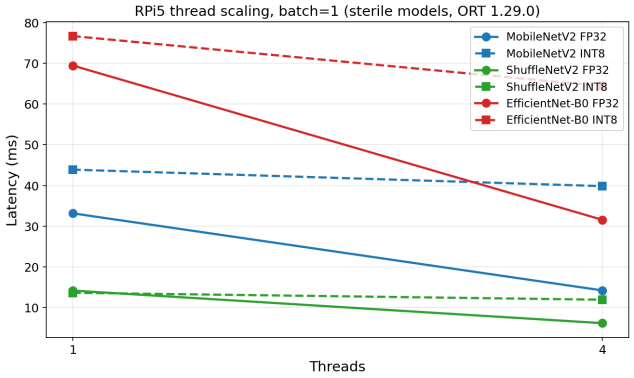


Fig. 2. RPi5 thread scaling at batch=1 (sterile models). FP32 (solid) scales from 1 to 4 threads; INT8 (dashed) is flat and slower than FP32 at 4 threads, except ShuffleNetV2 at 1 thread where INT8 is slightly faster.

On CPUExecutionProvider, the quantized graphs contain QuantizeLinear/DequantizeLinear (static QDQ) or DynamicQuantizeLinear+ConvInteger+Mul+Cast (dynamic) operators around the matmul. Because operator presence alone does not prove which kernels execute, we attribute the added cost cautiously: the pattern is consistent with quant/dequant and dispatch overhead without a compensating matmul saving on this stack (executed-kernel evidence in Table VIII). Cost grows with the number of quantize boundaries: depthwise separable blocks (MobileNetV2) create many small matmuls and slow down most on x86 (16.8×), while homogeneous channel-shuffle blocks (ShuffleNetV2) slow down least (3.0×). On RPi5 at 4 threads the ranking compresses (2.80/2.04/1.93×). Node expansion measured on the sterile ONNX graphs matches: MobileNetV2 170→487 (2.86×), EfficientNet-B0 239→730 (3.05×), ShuffleNetV2 902→1240 (1.37×). Flat thread scaling for INT8 is consistent with a poorly parallelized quant/dequant path unlike compute-bound FP32 GEMM; we state this as a hypothesis since memory-boundedness was not directly measured (no energy/power telemetry).

Model file size shrinks 3.3–3.7× with INT8 (e.g., 8.48→2.30 MB for MobileNetV2). On the sterile RPi5 sweep (batch=1 only), runtime RSS is similar across quantizations for MobileNetV2 (∼99–108 MB) and ShuffleNetV2 (∼111–112 MB), so INT8 does not save RAM at inference on this stack—a practical caveat for memory claims. We do not claim batch-scaling behavior for the sterile models; only batch=1 was measured.

TABLE VIII
LEVEL-2 PROVENANCE: NODE EXPANSION ON STERILE ONNX GRAPHS (DYNAMIC PTQ EXPANDS TO DYNAMICQUANTIZELINEAR + CONVINTEGER + MUL PATTERN, NO STATIC Q/DQ NODES) WITH ORT KERNEL QUANT OVERHEAD FROM RPI5 PROFILING (10 RUNS, BATCH=1, 1 THREAD). OVERHEAD CORRELATES WITH EXPANSION AND SLOWDOWN.

Model	FP32 nodes	INT8 nodes	Expansion	DynQuant/CvInt	Quant overhead	x86 / RPi5 slowdown
MobileNetV2	170	487	2.86×	53 / 52	29.2%	16.8× / 2.80×
ShuffleNetV2	902	1240	1.37×	54 / 56	20.7%	3.0× / 1.93×
EfficientNet-B0	239	730	3.05×	82 / 81	24.9%	13.1× / 2.04×

Measured bottleneck (Level-2). Table VIII quantifies why INT8 is slower. Dynamic PTQ does not insert static QuantizeLinear/DequantizeLinear nodes; it expands each Conv (52–81) into a DynamicQuantizeLinear + ConvInteger + Mul (scale) + Cast pattern, so node count grows by 1.37–3.05× on the sterile graphs. ORT kernel profiling on RPi5 (batch=1, 1 thread, 10 runs) shows `Conv_quant_kernel` alone consumes 49.5–55.1% of total time, and all quant-related kernels (QuantizeLinear, scale Mul, Cast, bias-add) add 20.7–29.2% overhead, while the final `Gemm_MatMul_quant` is $< 0.1\%$. The ranking matches the slowdown ranking: MobileNetV2 (29.2% overhead, 2.86× expansion) suffers most on x86, ShuffleNetV2 (20.7%, 1.37×) least. Artifact `level2_profiling_artifact.json` contains per-operator μs and full op counts (reproducible), complementing TinyML system studies [17].

D. Accuracy-Latency Tradeoff

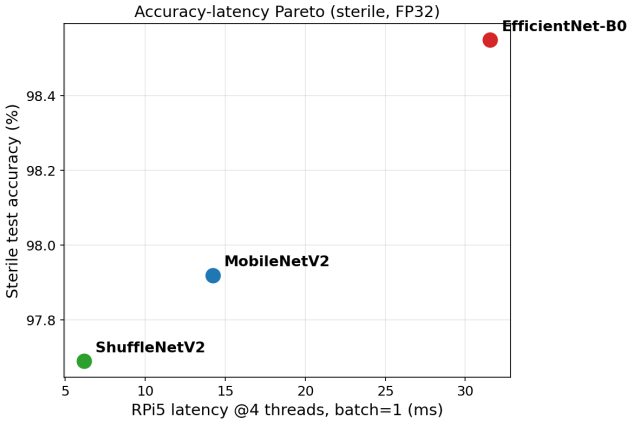


Fig. 3. Accuracy-latency tradeoff (sterile split, FP32). ShuffleNetV2 is fastest (6.19 ms RPi5 @4 threads, 97.69%); EfficientNet-B0 is most accurate (98.55%) at 31.56 ms; MobileNetV2 is balanced (97.92%, 14.23 ms).

The accuracy-latency plot (Fig. 3) shows that the fastest model is ShuffleNetV2 FP32 at 6.19 ms RPi5 (161.5 FPS), the most accurate is EfficientNet-B0 FP32 at 98.55%, and MobileNetV2 FP32 is the balanced middle (97.92%, 14.23 ms). The gap between best and worst FP32 accuracy is only 0.86pp (97.69% versus 98.55%); hardware constraints and quantization scheme should drive model choice for deployment. By "balanced" we mean MobileNetV2 FP32 sits between the latency extremes (14.23 ms vs 6.19 ms ShuffleNetV2 and 31.56 ms EfficientNet-B0 at 4 threads) while staying within 0.63pp of the best accuracy. All latency numbers are means of 200 runs after 10 warmup iterations (batch 1, model-only). On x86, ShuffleNetV2 FP32 (4.15 ms) is slower than MobileNetV2 FP32 (2.40 ms) but the order flips on RPi5 (6.19 ms vs 14.23 ms at 4 threads); reference and deployment hardware rank architectures differently, which is why deployment claims rest on RPi5 only.

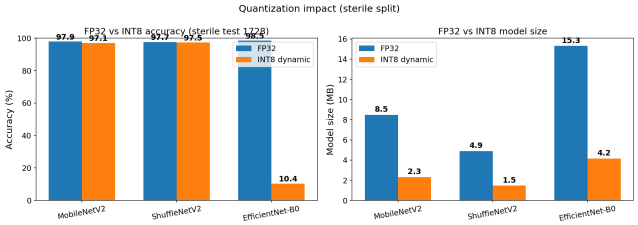


Fig. 4. Quantization impact on the sterile split: accuracy is preserved for MobileNetV2 and ShuffleNetV2 but collapses for EfficientNet-B0 under dynamic INT8, while model size shrinks 3.3–3.7× for all three.

E. Interpretability Note

Grad-CAM [15] was inspected during development as a qualitative sanity check on MobileNetV2; attention concentrates on color-gradient regions. We do not report it as a finding: a single-architecture visualization does not support cross-architecture or architectural claims, and quantization sensitivity is diagnosed quantitatively by the per-tensor versus per-channel experiment above.

F. Comparison with Related Work

TABLE IX
CONTEXTUAL COMPARISON WITH EXISTING WORK (DIFFERENT PROTOCOLS; NOT HEAD-TO-HEAD)

Work	Dataset (protocol differs)	Best Model	Acc. (%)	Edge Analysis
Dhungana *et al.* [1]	Machuve	EB0	99.1	Web
Tasdelen *et al.* [3]	Machuve	MN2	97.1	—
Degu *et al.* [4]	Machuve	R50	98.7	Mobile
Ours	**Machuve**	**EB0**	**98.6**	**+INT8+diagnosis**

Our accuracy (98.55%) is competitive with prior results on the same dataset family; Table IX is a contextual comparison across different evaluation protocols (splits, dedup, metrics), not a head-to-head benchmark. Our added analysis in quantization diagnosis and accuracy-latency tradeoff characterization under INT8 is absent from prior work.

V. CONCLUSION

This paper characterizes lightweight CNN architectures with INT8 quantization for poultry fecal disease detection. Our key findings are:

All three architectures exceed 97% FP32 accuracy with ImageNet transfer learning on the sterile split, with EfficientNet-B0 at 98.55%. Dynamic INT8 PTQ preserves

MobileNetV2 (97.11%) and ShuffleNetV2 (97.45%) but collapses EfficientNet-B0 to 10.42%; static per-channel quantization recovers EfficientNet-B0 to 94.56%, and selective per-family ablation confirms the cause: per-channel scaling on depthwise layers alone recovers 93.46%, while per-channel scaling on every family except depthwise collapses to 8.33%. On x86 reference hardware, INT8 increases latency by 3.0–16.8× (ShuffleNetV2 least, MobileNetV2 most); on RPi5 INT8 is 1.9–2.8× slower at 4 threads and scales by only 1.10–1.19× from 1 to 4 threads versus 2.20–2.33× for FP32. ShuffleNetV2 degrades least (3.0× on x86, 1.93× on RPi5) due to its homogeneous design; notably ShuffleNetV2 INT8 at 1 thread is the only configuration faster than its FP32 counterpart (13.65 vs 14.19 ms). On RPi5, batch 1 is the measured operating point and INT8 size reduction (3.3–3.7×) does not reduce runtime RSS. The tradeoff analysis identifies ShuffleNetV2 FP32 for the lowest RPi5 latency (6.19 ms at 4 threads) and MobileNetV2 FP32 as balanced (14.23 ms, 97.92%).

These findings enable practitioners to make informed deployment decisions based on their specific hardware constraints. This work characterized RPi5 deployment for one board, one runtime build, and one dataset; extending the selective-ablation protocol to MobileNetV2 and ShuffleNetV2 would test whether the same depthwise granularity threshold explains why they survive per-tensor quantization, and quantization-aware training against this PTQ diagnosis is the natural next step. Energy measurement on RPi5 and class-imbalance mitigation for the underrepresented NCD class remain open.

Code and sterile-split artifacts (dedup audit, split manifests, eval JSONs, benchmark logs, plotting scripts) are available at https://github.com/jaweed3/paper_poultry_p2/tree/post30-gate0-dedup.

REFERENCES

[1] A. Dhungana, X. Yang, B. Paneru, S. Dahal, G. Lu, and L. Chai, "An integrated deep learning approach for poultry disease detection and classification based on analysis of chicken manure images," *AgriEngineering*, vol. 7, no. 9, p. 278, 2025. DOI: 10.3390/agriengineering7090278.

[2] D. Machuve *et al.*, "Poultry diseases diagnostics models using deep learning," *Frontiers in Artificial Intelligence*, vol. 5, p. 733345, 2022. DOI: 10.3389/frai.2022.733345.

[3] A. Tasdelen and Y. Arslan, "Detection of high-risk diseases in poultry feces through transfer learning," *Engineering Science and Technology, an International Journal*, vol. 64, p. 102002, 2025. DOI: 10.1016/j.jestch.2025.102002.

[4] M. Z. Degu and G. L. Simegn, "Smartphone based detection and classification of poultry diseases from chicken fecal images using deep learning techniques," *Smart Agricultural Technology*, vol. 4, p. 100221, 2023. DOI: 10.1016/j.atech.2023.100221.

[5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE CVPR*, 2018, pp. 4510–4520.

[6] N. Ma, X. Zhang, H.-T. Huang, and J. Sun, "ShuffleNetV2: Practical guidelines for efficient CNN architecture design," in *Proc. ECCV*, 2018, pp. 116–131.

[7] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. ICML*, 2019, pp. 6105–6114.

[8] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE CVPR*, 2018, pp. 2704–2713.

[9] T.-Y. Chi, "FecalFed: Privacy-preserving poultry disease detection via federated learning," *arXiv:2604.00559*, 2026.

[10] U.S. Department of Agriculture, National Agricultural Statistics Service, "Poultry production and value: 2023 summary," Apr. 2024. [Online]. Available: https://www.nass.usda.gov/Publications/Todays_Reports/reports/plyaan24.pdf

[11] T.-M. Hoang *et al.*, "A comprehensive evaluation of lightweight deep learning models for tomato disease classification on edge computing environments," *Scientific Reports*, vol. 16, p. 12320, 2026. DOI: 10.1038/s41598-026-42439-6.

[12] Microsoft, "Quantize ONNX models," ONNX Runtime Documentation, 2024. [Online]. Available: https://onnxruntime.ai/docs/performance/quantization.html

[13] D. Machuve *et al.*, "Machine learning dataset for poultry diseases diagnostics," Zenodo, 2022. DOI: 10.5281/zenodo.4628934.

[14] M. Nagel *et al.*, "A white paper on neural network quantization," *arXiv:2106.08295*, 2021.

[15] R. R. Selvaraju *et al.*, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626.

[16] J. Deng *et al.*, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 248–255.

[17] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. Machine Learning and Systems (MLSys)*, 2021, pp. 800–815.